

★ ★ ★ DEFECT TRACKING ★ ★ ★

(30% of Interview Questions comes from this topic)

Defect : If a feature is not working according to the requirement, it is called a defect.
Deviation from requirement specification is called as defect.

INTERVIEW QUESTIONS

1) What is the difference b/w defect, bug, error and failure?

Ans)

BUG is informal name given to the defect.

ERROR – it is a mistake done in the program because of which we are not able to compile or run the program

FAILURE – defect leads to failure or defect causes failure. Chances are there 1 defect might lead to 1 failure or multiple failures.

Take the example of Amount Transfer feature not working for end users when end user tries to transfer money – submit button is not working. Thus, this is a failure.

DEFECT – The variation between the actual results and expected results is known as defect.

A bug occurs **only** because of the following reasons,

- **Wrong implementation** :- Here, wrong implementations means coding. *For ex*, in an application – when you click on “SALES” link – it goes to “PURCHASE” page – this occurs because of wrong coding. Thus, this is a bug.
- **Missing implementation** :- We may not have developed the code only for that feature. *For ex*, open the application – “SALES” link is not there only – that means feature has not been developed only – this is a bug.
- **Extra implementation** :- The developer develops extra feature which is not needed and not there in the requirements also. *For ex*, consider the below application

....	SUBMIT CANCEL HELP
--	--

In the above application – requirements say only to develop “submit” and “cancel” button – but the developer also develops “help” button which is not there in the requirements.

- If we develop extra features not there in the requirements, it leads to unnecessary extra effort
- Chances are there that adding extra features might affect other features.

Now, let's understand "**WHAT IS DEFECT TRACKING**"

Developer develops the product – test engineer starts testing the product – he finds a defect – now the TE must send the defect to the development team.

He prepares a defect report – and sends a mail to the Development lead saying "bug open".

Development lead looks at the mail and at the bug – and by looking at the bug – he comes to know to which development engineer developed that feature which had a bug – and sends the defect report to that particular developer and says "bug assigned".

The development engineer fixes the bug – and sends a mail to the test engineer saying "bug fixed" – he also "cc mail" to the development lead.

Now the TE takes the new build in which the bug is fixed – and if the bug is really fixed – then sends a mail to the developer saying "bug closed" and also "cc mail" to the development lead.

Every bug will have an unique number.

If the defect is still there – it will be sent back as "bug reopen".

We should also send a copy of the defect report to the TL. Why do we do this ? Because,

- He should be aware of all the issues that are there in the project
- To get visibility (i.e, he should know that we are working)

90% of projects – we don't take permission from Test Lead to send bugs to development team.

Around 10% of projects, we take permission because,

- **Customer is new** – for ex, Reliq has a testing team which is testing a product developed by Vodafone developers. We cant send all sorts of major, minor and critical bugs to their development team. So test lead first approves the defect and then sends it to the development team saying it's a valid bug.
- **When we are new to the project**

When we should send defects to development team? – as soon as we catch the defect, we send it to development team.

Why do we send it immediately?

- Otherwise someone else will send the defect (common features)
- Development team will have sufficient time to fix the bug if we send the bug asap.

NOTE

For which and all types of testing do we write test cases ?

Smoke Testing – here, we are testing basic features only. So we can pull out some test cases which has all the basic features – so we don't have to write test cases for this.

Functional Testing – yes, we write test cases

Integration Testing – yes, we write test cases

System Testing – yes, we write test cases

Acceptance Testing – yes, customer may write test cases

Compatibility Testing – we don't write because the same test cases as above are used for testing on various platforms

Adhoc Testing – we dnt write because they are "on the fly" or "random" ideas/scenarios. However, if we find critical bug, then we convert that scenario into test case.

Performance Testing – we may not write test cases because we do this using tool.

Usability Testing – we use standard checklist. We don't write test cases because we are just testing the look and feel of the application here.

Accessibility Testing – here also, we use checklist

Reliability Testing – we don't write manual test cases and we use automation tool

Regression Testing – Yes – we have FT, IT, ST test cases. No – because we use the same test cases.

Recovery Testing – Yes, we write test cases. We check how the product recovers from crashes

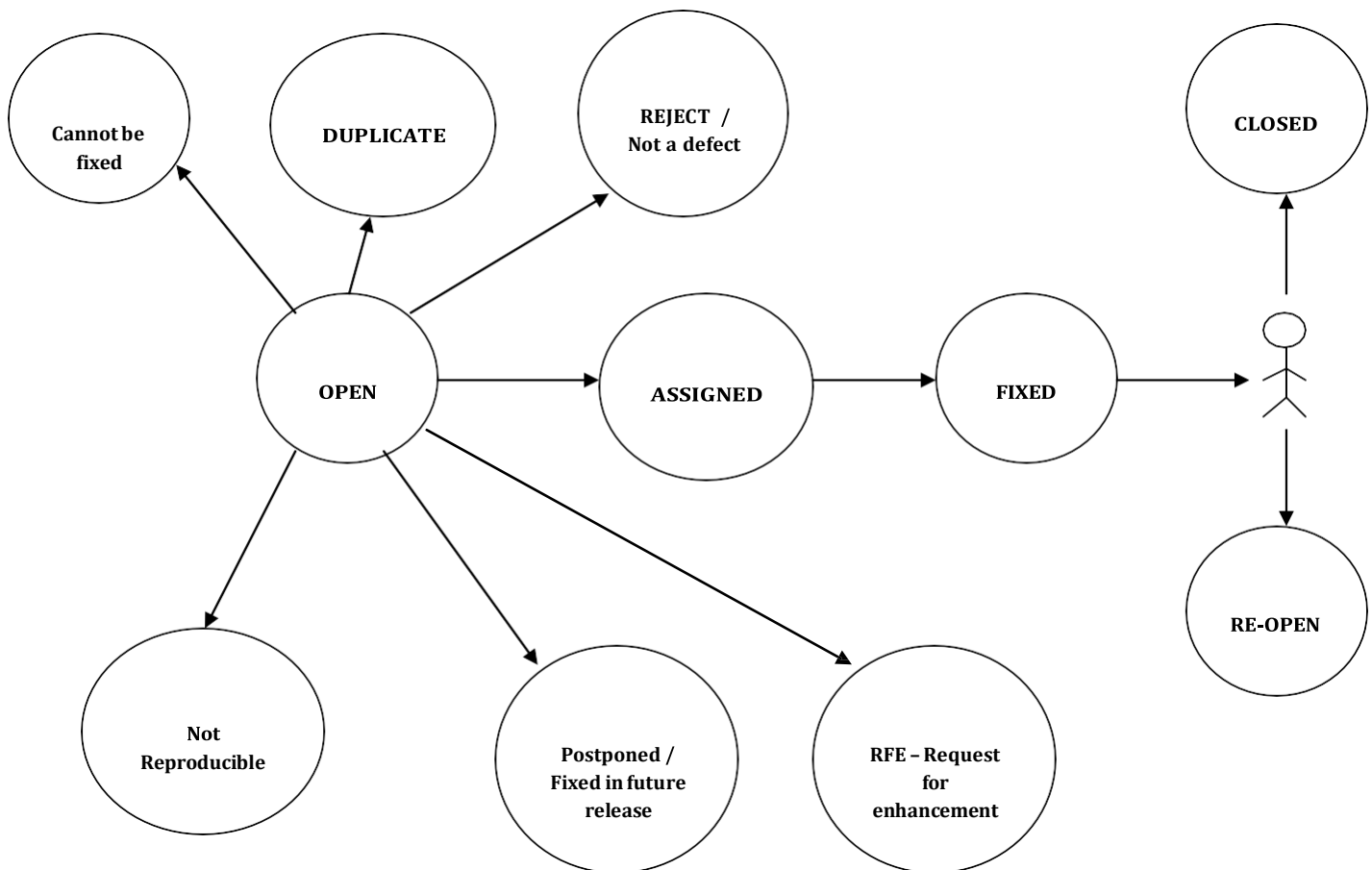
Security Testing – yes, we write

Globalization Testing,

L10N testing – yes, we write

I18N testing – yes, we write

★★ DEFECT LIFE CYCLE ★★



Customer gives requirements – developers are developing the s/w – testing team is writing test cases looking at the requirements

Developer develops the product – test engineer starts testing the product – he finds a defect – now the TE must send the defect to the development team.

He prepares a defect report – and sends a mail to the Development lead saying “bug open”.

Development lead looks at the mail and at the bug – and by looking at the bug – he comes to know to which development engineer developed that feature which had a bug – and sends the defect report to that particular developer and says “bug assigned”.

The development engineer fixes the bug – and sends a mail to the test engineer saying “bug fixed” – he also “cc mail” to the development lead.

Now the TE takes the new build in which the bug is fixed and re-tests it again – and if the bug is really fixed – then sends a mail to the developer saying “bug closed” and also “cc mail” to the development lead.

Every bug will have an unique number.

If the defect is still there – it will be sent back as “bug reopen”.

“REJECT” BUG

Now, when the TE sends a defect report – the Development Lead will look at it and **reject the bug**.

Bug is rejected because,

1) Misunderstanding of requirements

2) While installing or configuring the product – wrongly configured or installed the product and we found a bug in the product – send it to development team – developer says “reject” because he looks at the defect report and comes to know that the Testing team has not installed the product correctly.

3) Referring to old requirements

Chances are there that Testing team may be referring to old requirements while testing the product.

For ex, - in the 1st build – developers have given the product for testing in which a **sales link button** is there, test engineer does testing and reports any defects – in the 2nd build, customer has given a requirement change to the development team where-in he has asked them to remove the **sales link button** – but the testing team is not aware of this change – so when the development team gives the new build – obviously they would have removed the **sales button** – test engineer when he looks at the feature – he is referring to old requirements which has **sales link button** – but in the new requirements it has been removed which the test engineer is not aware of – so he reports a bug – development team then reply back saying “refer to new requirements in which sales link button has been removed”.

4) Because of extra features

Consider the example and figure shown in Pg 130.

Here, “**help**” button is a feature which has been developed by the developer as an extra feature not part of he requirements – obviously when the TE looks at the application and requirements, since Help feature is not there – he reports it as a bug – the developer rejects it saying “it is an extra feature and that he wont test it” – Test Engineer replies back by re-opening the bug saying “update the requirements” – updating the requirements requires lot of running around for the development team, talking to the customer and all that – so he either removes it or talks to the customer.

“DUPLICATE” BUG

The TE finds a bug and sends a defect report and also assigns the bug report a number – let’s say he sends a bug report numbered as Bug 25 – now the developer replies back by saying “Bug 25 is duplicate of Bug 10” i.e, another Test engineer has already sent that bug earlier and it is being fixed.

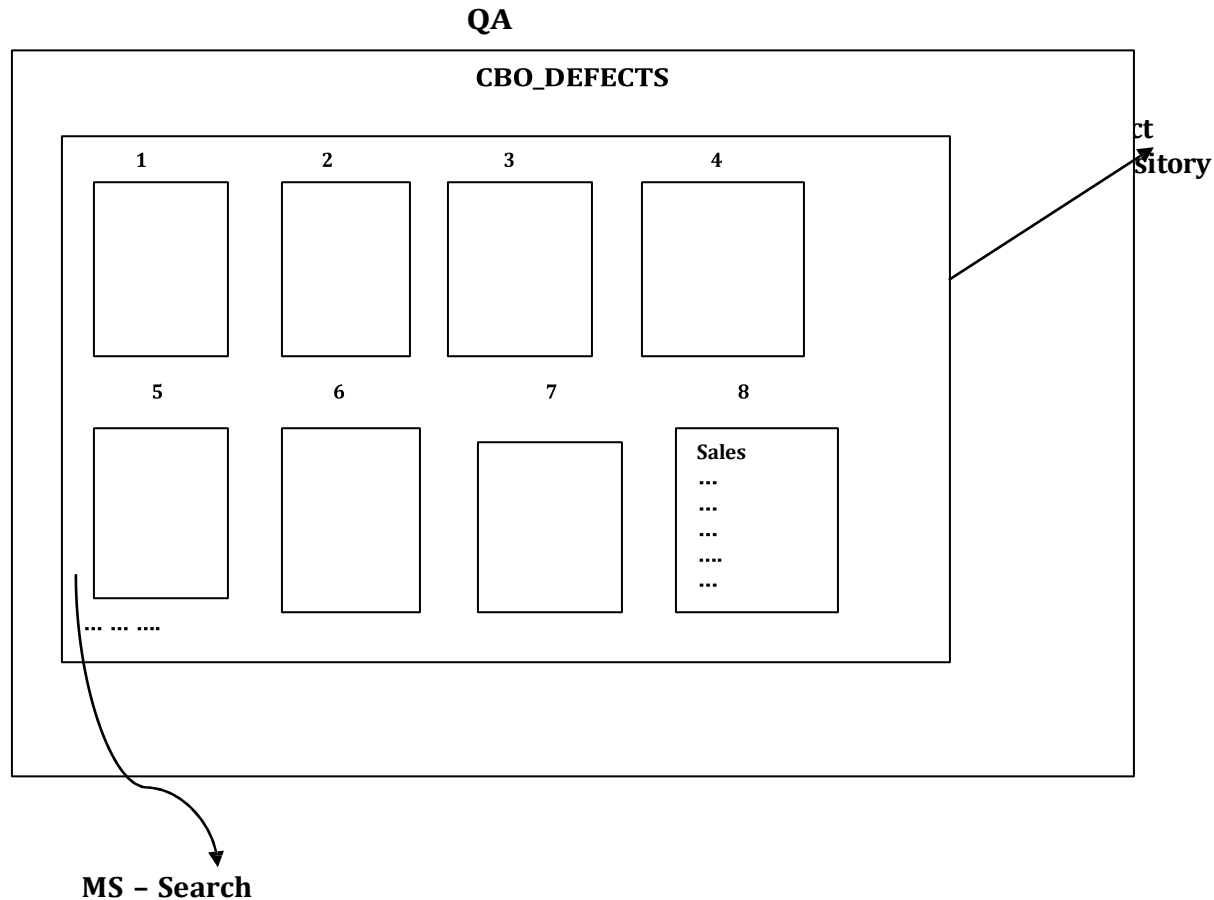
Why do we get duplicate bugs?

- *Because of common features* – Lets say Test Engineer A and B are testing an application – Test Engineer A and B to test their features must Login to the application using valid username and password before they can go deep into the application and start testing their features – A enters valid username and password and clicks on Login button – it goes to a blank page – this is a bug – so A

prepares a defect report for this bug and sends it to developer – After sometime, B also tries to login and finds the same bug – he also prepares a defect report and sends it to developer – but developer sends back defect report of B saying “it is a duplicate”.

- *B finds bug in A's module* – let's say A and B are testing their respective features – A is doing functional testing and integration testing on all his features – Similarly B is also testing all his features – In one scenario, B needs to go to A's feature(say m12) and send data to B's feature(say m23) – when B clicks on m12, he finds a bug – although its feature of A, he can still prepare a defect report and send it to developers – and B sends the defect report to development team – now, A after testing all his features comes to his feature m12 – he also finds the same defect and sends a defect report – but the developers send it back saying “it is duplicate”.

How to avoid duplicate bugs?



Let us consider Test Engineer A is testing Sales module and finds a defect where-in – he enters all the information and then when he clicks on Submit button, it is going to Blank page – he prepares a defect report in which he explains the defect.

Now, before he sends it to developer – A logs into a server named QA – there, he goes into a folder called “Defect Repository” which has all the defect reports prepared by TE(s) and sent to developers and clicks on MS-Search and gives Sales as the search keyword and hits on search button – now, it starts looking inside the folder if there are any defect report which has Sales in it – if the search results in No files – then A will first copy-paste his defect report in the defect repository and then send the defect report to the development team who then assign it to the development engineer.

Now, after a few days – another TE B finds the same defect and he also prepares a report – but before he sends it to development team. He first logs onto QA and Defect Repository and searches for Sales keyword – he finds the report which was earlier sent by TE A – so he doesn't send the defect to development lead.

But, always – the TE if he finds a defect similar to his if the Search operation yields some results – before he concludes that his defect has already been sent – he must first read the defect report and ascertain whether it is the same bug which he has found – chances are there that it might have the keyword Sales, but it is about some other defect in Sales feature.

For ex, - TE A is testing password text field – he finds that it is accepting 12 characters also (when requirement says “it must accept only 10characters”) – that's a bug. So he prepares a defect report which has the keyword password and stores it in the defect repository and also sends it to Dev team. Now, another TE B is also testing password field – he finds there is a bug where-in password text field is also accepting blank space character – before he prepares a report, he goes to the defect repository and searches for Password keyword – there he finds the earlier report sent by A – but that's a different defect – he reads it and realizes that it's not the same defect which he has found – so, he prepares a defect report regarding the blank space acceptance in the password text field – and stores it in the defect repository and also sends it to the development team.

CANNOT BE FIXED

Chances are there – Test Engineer finds a bug and sends it to Development Lead – development lead looks at the bug and sends it back saying “cannot be fixed”.

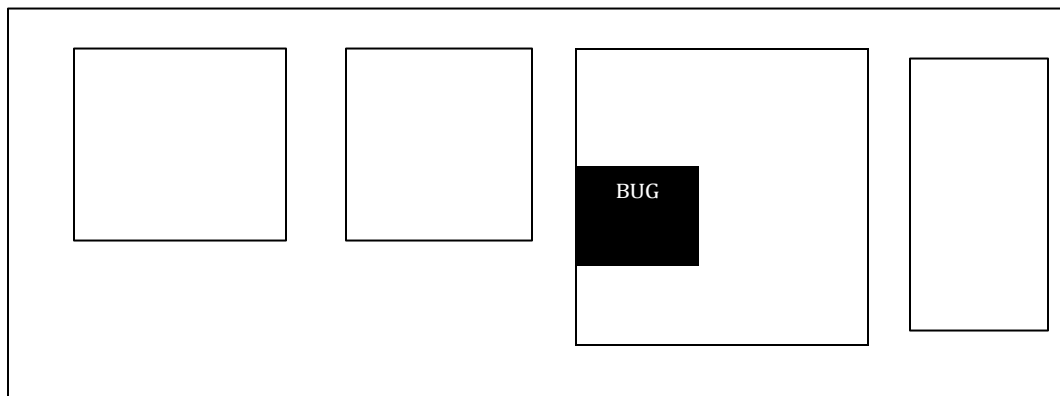
Why does this happen? – Because,

- Technology itself is not supporting i.e, programming language we are using itself is not having capability to solve the problem
- Whenever there is a bug in the root of the product. If it's a minor bug, then development lead says “cannot be fixed”. But, if it's a critical bug, then development lead cannot reject the bug
- If the cost of fixing the bug is more than the cost of the bug itself – cost of the bug means loss incurred because of the bug.

POSTPONED or Fixed in the next release

a) We find a bug during the end of the release (could be major or minor but cannot be critical) – developers won't have time to fix the bug – such a bug will be postponed and fixed in the next release and the bug status will be “open”

b)



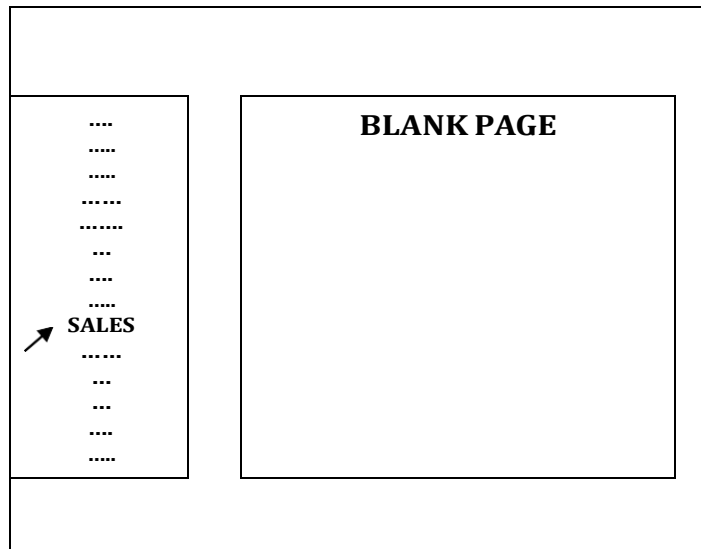
Developers have built the above application – Test Engineers are testing the application and they find a bug – now, the bug is sent to the development team.

Development team looks at the report – and replies back to the testing team saying “they will not fix the bug for now, as the customer is thinking of making some changes to that module or he might as well ask for a requirement change where-in the module itself might have to be removed” – so they won’t fix the bug for now – but if no requirement change happens, then they will fix the bug.

c) If the bug is minor and it is in feature exposed to internal users.

For ex, consider Citibank employee using a s/w where-in he is making keeping track of accounts and customer details – now, for example if the “Sort by name” feature is not working – it’s ok because it’s a minor bug because the development team can always fix the bug during the next release.

NOT REPRODUCIBLE

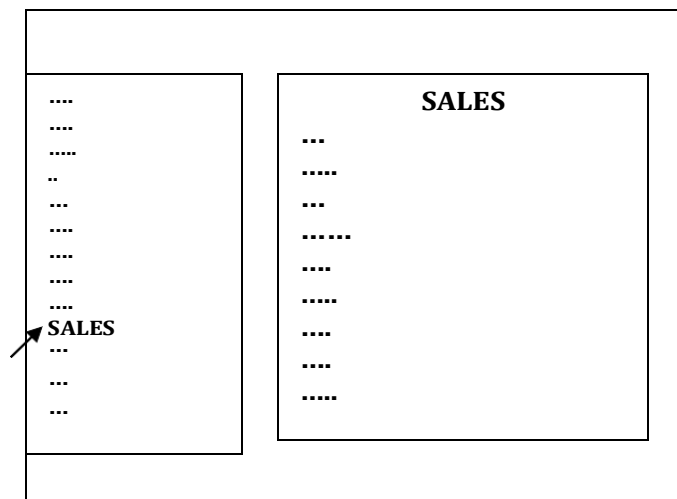


Open the application in Mozilla FireFox

- 1) Open browser and enter www.citibank.com and click on Go button
- 2) Login using valid username and password
- 3) In the homepage, click on “SALES link button”
- 4) It goes to blank page

DEFECT REPORT

Let us consider the above application – the TE is testing the above application in Mozilla Firefox– when he clicks on SALES link button – it goes to a blank page – this is a bug – the TE prepares a defect report and sends to the development team.



Open the application in Internet Explorer

The development lead looks at the defect report and opens the application in Internet Explorer – when he clicks on Sales link button, Sales page opens and the application works perfectly.

Then, what was the problem? – the answer is, the TE didn't specify in the defect report in which browser to open the application – so, the application didn't work for the TE in Mozilla FireFox – but, when the developer opened it in Internet Explorer, the application worked totally fine.

Since the developer is not able to see any defect – he replies back by saying “I am not able to find the defect” or “I am not able to reproduce the defect”.

Why we get “not reproducible” defects?

1. Because of platform mismatch
2. Because of improper defect report
3. Because of data mismatch
4. Because of build mismatch
5. Because of inconsistent defects

1) Because of platform mismatch

- Because of OS mismatch
- Because of browser mismatch
- Because of „version of browser“ mismatch
- Because of „settings of version“ mismatch

2) Because of improper defect report

Example to explain this - take figure and explanation in Pg 136 and 137. Here, the TE must have specified to open the application in Mozilla Firefox

Let us consider another example shown below to explain “improper defect report”

USER DETAILS

....

....

....

....

....

....

....

....
...
...
....
...
...
...

☒ REMEMBER PASSWORD

SUBMIT

CANCEL

1) Open browser and enter www.citibank.com and click on Go button

2) Enter user details and click on Submit button

3) It goes to blank page

DEFECT REPORT

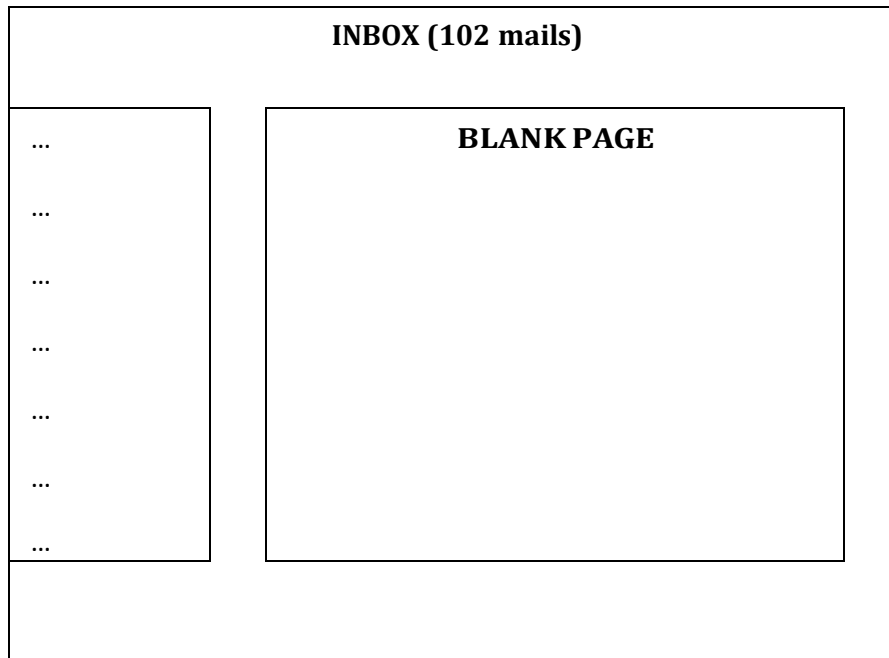
In the above application – TE is testing the above application – when he enters all the user details and selects checkbox and clicks on Submit button – it goes to blank page – but, when the TE does not select the

checkbox and clicks on Submit button, a new user account is created, i.e, it works fine. The TE prepares a defect report and sends it to development lead.

The Dev team looks at the report(as shown in box) – and tests the application – it is working absolutely fine. Then what happened? – Observe the defect report sent by the testing team – nowhere as he mentioned to “select checkbox and click on submit button” – thus because of the improper defect report – the development team is not able to reproduce the defect.

3) Because of data mismatch

Consider the example shown below,



Login as ABC and click on Inbox in homepage of ABC

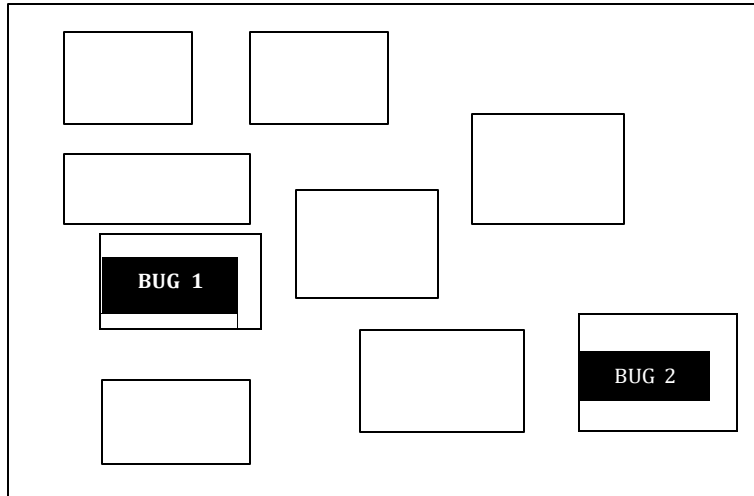
The above email application has been developed – the TE starts testing the above application – after testing and testing the above application using username ABC and valid password – there are almost 101 mails in ABC’s inbox – now, again when the TE opens the application and goes to test the application – no mails are being displayed when he clicks in Inbox and instead he gets a blank page. So – the defect he has found is that – when the number of mails in Inbox exceeds 100 mails , he gets a blank page – so he prepares a defect report as shown in the box below,

- 1) Open the browser and enter the following URL www.email.com
- 2) Login using valid username and password
- 3) Click on Inbox
- 4) Blank page is displayed

The development team looks at the defect report – he logs in to the application using username XYZ and clicks on Inbox – no blank page is displayed and the Inbox is working fine.

What happened here? – The TE should have mentioned in the defect report as “Login as ABC” or “Login to any mailbox which has more than 100mails”. Thus, the developer does not find any defects and thus he mentions it as “not reproducible defect”.

4) Because of build mismatch



Developer develops build 1 and gives it for testing – in Build 1, there are 2 defects – bug 1 and bug 2 – TE while testing finds bug1 and reports it to development team – development team when they fix bug1, bug2 automatically gets fixed – development team is developing build 2 and gives it to testing team for testing – testing team just when it finishes testing build1 – he finds Bug 2 and reports it to development team – the development team however do not find any defect as they are testing build 2 – so they report back saying “bug is not reproducible”.

5) Because of inconsistent defects

To explain this, let us consider an example : When TE is testing an email application – he composes mail in User A and sends it to User B – he then logs out from user A and logs into User B and checks B’s inbox – but no mail is there in the Inbox – thus he finds a defect – now, just to confirm the defect before sending a defect report – the TE again logs into User A and sends a mail to User B – he then logs out from User A and logs into User B and checks B’s inbox – but, this time the mail is there in B’s inbox!!

Thus, we don’t know when the defect comes and when the feature works fine. This is called inconsistent defect.

REQUEST FOR ENHANCEMENT (RFE)

Test engineer finds a bug and sends it to development team – when development team look at the report sent by the TE – They know it’s a bug, but they say its not a bug because its not part of the requirements.

Let us consider the **example** shown below,

The image shows a web form titled "CREATE USER". On the left side, there is a vertical list of input fields, with the first one labeled "CREATE USER" and an arrow pointing to it. To the right of this list, there are several more input fields. At the bottom of the form, there are two buttons: "SUBMIT" and "CANCEL".

In the above **example**, the TE is testing the above fields. After he enters all the data into the required fields, he realizes that he needs to clear all the data – but there is no **CLEAR button** – and he has to manually clear all the fields. He reports this as a bug – but the development team will look at the defect report and see that **clear button** is not mentioned in the requirements – so they don't reject or close the bug – instead they reply back to TE saying that it is RFE.

Developers agree that it is a bug – but will say that the customer have not specified these details and hence will give it a name as RFE.

If a defect is known as RFE, then we can bill the customer. Otherwise, development team needs to fix the bug

Development team always have a tendency to call a defect as RFE, so a TE needs to check the justification given by development team – if it is valid, then he will accept it as a RFE – but if it is not, then he will respond to the development team with proper justification.

INTERVIEW QUESTIONS & TIPS

1) Explain Defect Life Cycle (** VERY VERY IMPORTANT QUESTION ****)**

When TE is testing the product – he finds a defect – he prepares a defect report and creates a status as “OPEN” – and sends it to development lead – development lead looks through the defect report – and if the defect is valid – he then finds out who developed that module in which the defect has been found and assigns the defect to that development engineer – the development lead also changes the status of the defect report to “ASSIGNED” – the development engineer fixes the defect and changes the status to “FIXED” – the build is then sent to the TE for re-testing – the TE tests the fix – and if the defect has been really fixed – then he changes the status of the defect report to “CLOSED” – if the defect is still there, he changes the status to “RE-OPEN”.

Sometimes it so happens that TE finds a defect and sends it to development lead – development lead looks at the report and rejects the report and he explains why it has been rejected,
REJECT (explain)

DUPLICATE (explain Duplicate)

CANNOT BE FIXED (explain)

POSTPONED (explain)

NOT REPRODUCIBLE (explain)

RFE (explain)

DEFECT REPORT

Defect ID – it is an unique number given to the defect

Test Case Name – whenever we find a defect, we send the defect report and not the test case to the developer. For defect tracking, we only track the defect report and not the test case. Test case is only for reference for the TE. We always only send the defect report whenever we catch a bug.

When we are doing ad-hoc testing – no test case is written for ad-hoc testing because they are “out of the box” testing scenarios – if we find a critical bug – then we convert that ad-hoc scenario into a test case and send it to development team.

Given below is – how a defect report looks like – it is a MS – WORD file.

DEFECT ID :	BugID _ 1578		
RELEASE NAME :	Tiger		
BUILD ID :	B03		
MODULE NAME :	Sent Items		
STATUS :	Open		
	Assigned		
	Fixed		
	Closed		
	Duplicate		
	... etc etc		
SEVERITY :	Blocker OR Showstopper		
	Critical		
	Major		
	Minor		
PRIORITY :	High	P1	
	Medium	P2	
	Low	P3	
		P4	
TEST DATA :	username = abc123 , password = xyz123		
TEST ENVIRONMENT :	In Windows XP, Internet Explorer 7.0		
TEST CASE NAME :	Yahoo _ SI _ mail list		
FOUND BY :			

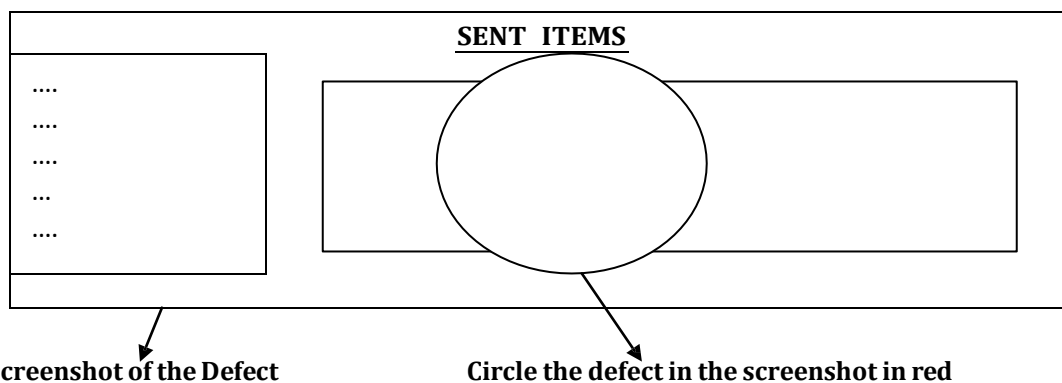
BRIEF DESCRIPTION : Sent mails are not there in the Sent Items page

DETAILED DESCRIPTION : Following are the steps to reproduce the defect

1. Open the browser and enter the test URL
2. Login with valid username and password
3. Click on “compose” link
4. Enter valid data into all fields and click on Submit button
5. Click on Sent Items link

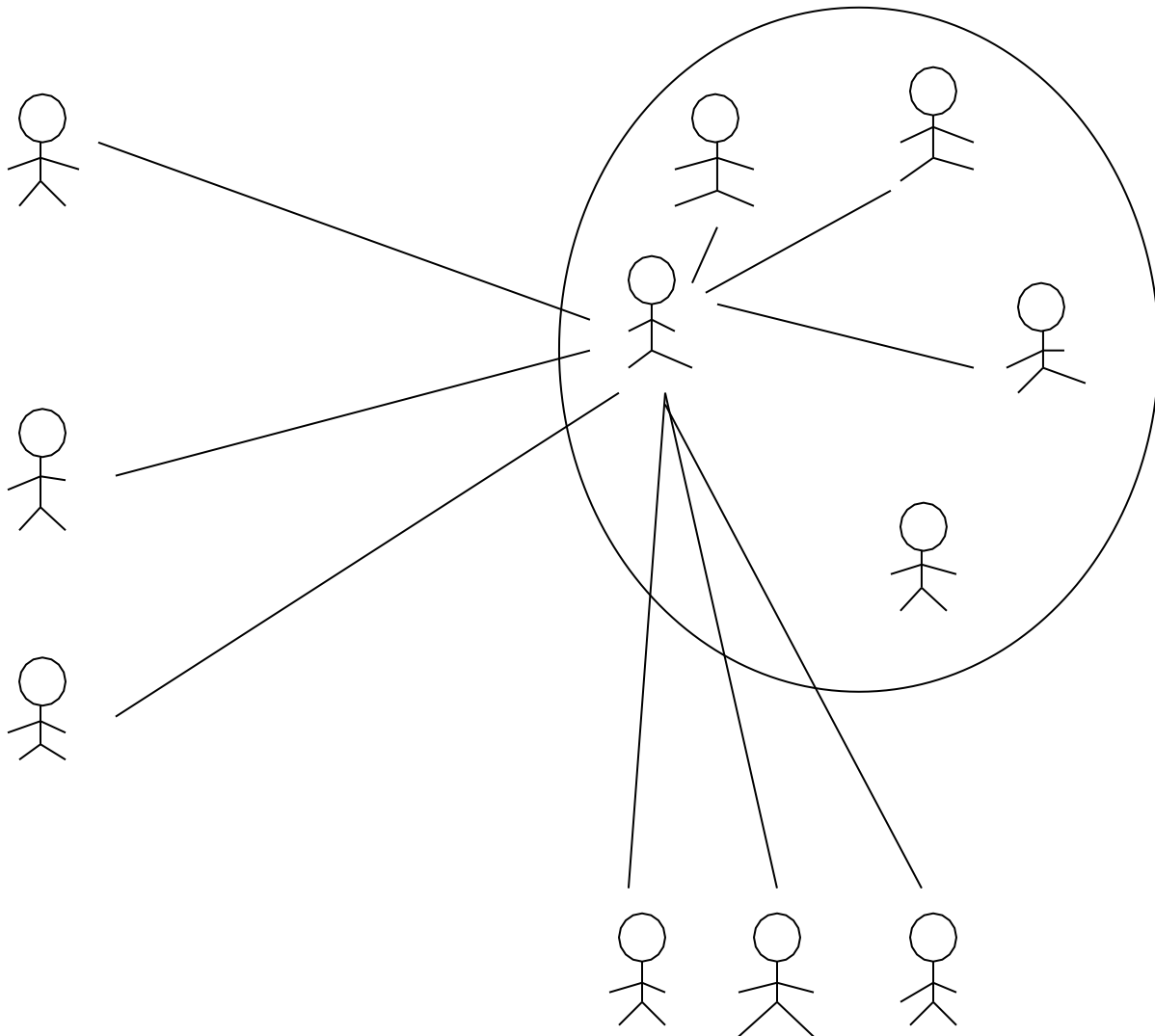
EXPECTED RESULT : Sent mail should be listed in Sent Items page

ACTUAL RESULT : Sent mail is not there in the Sent Items page.



INTERVIEW TIPS

Q) What are the attributes of a defect report ? (OR) What is the mode of communicating a defect ?
Ans) the answer for the above questions are – just explain the defect report.

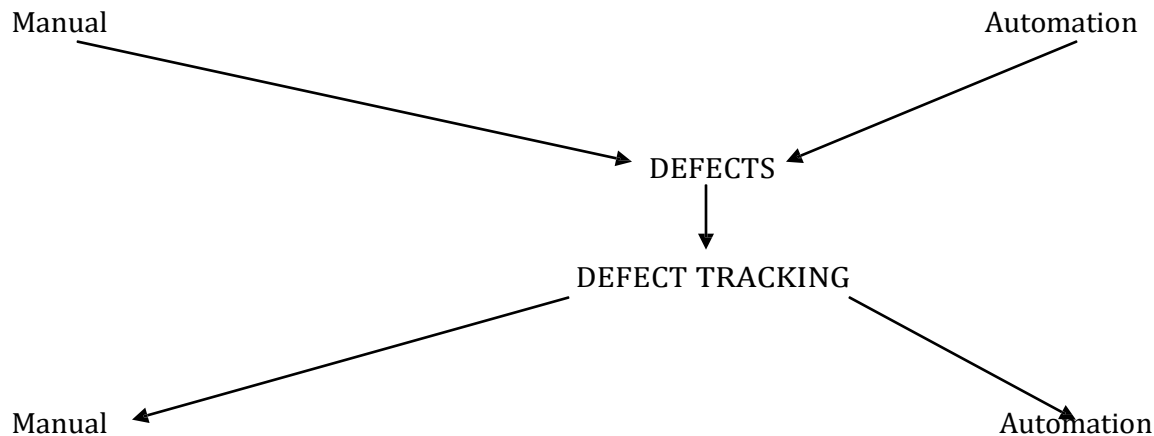
Defect Evaluation Team / Bug Counseling Team

Let us consider the above figure to explain about **Bug review meeting / Bug Counseling**

The Defect Evaluation Team (DET) consists of Development Lead, Development Manager, Senior Development Engineers and Test Lead.

They will have a group mail id – say – xyz_det@abc.com – Now, whenever TE finds a bug, he sends it to DET – In DET, only the Development Lead (DL) looks into the bug report and assigns the bug – although the mail goes to everybody in the DET, only DL will check it and assign the bug. This process continues from Monday morning upto Friday afternoon – by this time about 100 bugs have been reported by TE(s) – out of these 100 bugs, 60 have been assigned and the remaining 40 are pending.

Now, on Friday evening – DET team will have a meeting where they discuss about the pending bugs and whether to assign the bugs or reject the bugs – but for everything, they will have to give proper justification. This meeting is called Bug Review Meeting / Bug Counseling. By the end of the meeting (i.e, 1st cycle) – the status of all the pending bugs will be changed from pending to other status(es).



How to track a defect manually?

- 1) Find a defect
- 2) Ensure that it is not duplicate (i.e, verify it in Defect Repository)
- 3) Prepare defect report
- 4) Store it in defect repository
- 5) Send it to development team
- 6) Manage defect life cycle (i.e, keep updating the status)

Tracking of defects using Automation (i.e. Defect Tracking Tool)

The various tools available are,

- Bugzilla
- Mantis
- Rational Clear Quest
- TeleLogic
- Bug_track
- QC – Quality Center – it is a test management tool – a part of it is used to track the defects.

There are **2 categories of tools**,

- 1) Service based – In service based companies – they have many projects of different clients – each project will have a different defect tracking tool
- 2) Product based – in product based companies – they use only one defect tracking tool.

Now, let us see **how the defect tracking tool works**,

http : // www.hp.com

USERNAME

PASSWORD

LOGIN CANCEL

Figure 1

Welcome to YahooMail

.....

.....

.....

.....

.....

.....

.....

➡ Log a defect / New defect

➡ Defect ID

➡ Advanced Search

Figure 2

Assume that you are working as a TE in HP and that you are working on YahooMail project. We enter [http : // www.hp.com](http://www.hp.com) as the URL in the browser and give valid username and password and click on **login** button.

The username of your"s can be obtained by the following two ways,

- Use **employee id** as username along with the standard password to access the system
- Or send a request to the admin of tool and ask for username and password.

When we login, we go to the page shown in figure 2. We then click on **log a defect**. When we click on this link, we get the following page shown below,

<u>RELEASE NAME</u>	TIGER CHEETAH DOLPHIN
<u>BUILD ID</u>	B01 B02 B03
<u>MODULE NAME</u>	INBOX SENT ITEMS DRAFTS Etc
<u>STATUS</u>	OPEN ASSIGNED FIXED ... etc etc
TO	
<u>SEVERITY</u>	BLOCKER CRITICAL MAJOR MINOR
<u>PRIORITY</u>	HIGH P1 MEDIUM P2 LOW P3 P4
<u>TEST DATA</u>	
<u>TEST CASE NAME</u>	
<u>TEST ENVIRONMENT</u>	
<u>BRIEF DESCRIPTION</u>	
<u>DETAILED DESCRIPTION</u>	
<u>COMMENTS</u>	ATTACH
SUBMIT CLEAR CANCEL HISTORY	

When we click on **log a defect** link, we get the following page as shown above.

We enter all the details and then when we click on **submit button**, the following happens,

- It generates a bug ID automatically, **ex** – bugID_1578
- It stores the data in the database
- It automatically sends the mail to the Development Lead and also the copy of the mail goes to the TE himself.

Development Lead opens the mail – he logs in to the tool using his username and password – then he gets the **welcome** page – he then enters the BugID into the **defect id** text box and clicks on **search** button

When he clicks on **search** button – he gets the particular bug report. He looks at the report and after checking it's a valid bug, he then **changes the status in the report to „assigned“** and when he does that – he gets a **TO text field** as shown in the figure above – there he enters the email id of the concerned developer who has to fix that bug – and then when he clicks on **submit button** – whatever changes he has made is stored in the database – and the mail goes to the concerned TE, Developer and DL himself.

When the particular developer gets the the BugID, then he searches for the bug using the bugID and after seeing what's the bug – he goes to his module and fixes the bug.

When developer fixes the bug – he goes and changes the status to **fixed** and again he submits it. This is also saved in database. Then mail goes to TE, DL and to the developer himself.

TE now opens the same bugID and checks the status whether it is fixed or not – if it is changed to **fixed** – then he re-tests the bug and if it is really fixed, then he goes to status and changes it to **closed** – if the bug is not fixed, then he changes the status to **reopen**.

This comprises the entire defect life cycle.

As soon as TE submits the bug report, then he or anybody cannot change two fields in the entire report – they are „brief description“ and „detailed description“.

If any changes are to be made in these fields – then they must be mentioned in the **comments** field.

If a new defect is found within a old defect within a short period (say 10days), then instead of opening a new defect – he can open the old defect and login the new defect in the **comments** page.

How to check for duplicate bugs ?

When developer changes the status to **duplicate**, then the TE should check whether the previous bug & the sent bug is same or not.

To check whether its duplicate or not, click on **advance search** & get it confirmed if the bug is duplicate or not. If it's not duplicate, then TE should give proper justification.

Click on **Advanced Search** (see figure next page)

To avoid duplicate bugs, go to Advanced Search.

When ever we catch a bug – Before logging the bug to developer for fixing – first go & check whether it is logged before or not. To do so, click on **Advanced Search** & enter the data in the **text** field & click on **Search**. You will get the bug ID(S). if you enter „**password**“ & search, then it will give you different bug ID having **password** text. We must go & check it & log it for fixing if its not logged before.

Defect found by Testing team should never be closed just like that by the Development team. TE looks the product from customer point of view – so if a developer says it's a minor bug, testing team always considers it as major bug.

SEARCH BY**Release Name****Module Name****Status****Priority****Severity****Found By****Text****SEARCH****CANCEL****RESULT SHEET**

<u>Release Name</u>	<u>BugID</u>	<u>STATUS</u>	<u>Severity</u>	<u>Priority</u>
TIGER	BugID 1578	OPEN	...	...
TIGER	BugID 1890	CLOSED	...	...
TIGER	BugID 1235	ASSIGNED	...	...

SEVERITY of a Bug**Severity** is impact of the bug on customer's business.

- **Critical** – A major issue where a large piece of functionality or major system component is completely broken. There is no work around & testing cannot continue.
- **Major** – A major issue where a large piece of functionality or major system component is not working properly. There is a work around, however & testing can continue.
- **Minor** – A minor issue that imposes some loss of functionality, but for which there is an acceptable & easily reproducible workaround. Testing can proceed without interruption.

Blocker Defect

There are 2 types in **blocker** defect,

- ✓ **Major flow is not working** – Login or signup itself is not working in CitiBank application
- ✓ **Major feature is not working** – Login to CitiBank. Amount Transfer is not working

PRIORITY of a Bug

It is the importance to fix the bug (OR) how soon the defect should be fixed (OR) which are the defects to be fixed first.

- **High** – This has a major impact on the customer. This must be fixed immediately.
- **Medium** – This has a major impact on the customer. The problem should be fixed before release of the current version in development
- **Low** – This has a minor impact on the customer. The flow should be fixed if there is time, but it can be deferred with the next release.

Development team will fix the **high priority** defects first rather than of **high severity**.

Generally, severity is assigned by Tester / Test Lead & priority is assigned by Developer/Team Lead/Project Lead.

EXTRA(s) Information

Why does a software have bugs ?

- ✓ Mis-communication OR No communication -> as to specific of what an application should or shouldn't do(the application's requirements)
- ✓ Software complexity
- ✓ Programming errors
- ✓ Changing requirements
- ✓ Time pressure

When do we stop the testing ?

- ≈ When the time span is less, then we test the important features & we stop it
- ≈ Budget
- ≈ When the functionality of the application is stable
- ≈ When the basic feature itself is not working correctly

In the last 2 days of release, the code will be freezed. When the code is freezed, the defect found will be fixed in later stages.

Why do we do testing ?

- ✓ To ensure the quality
- ✓ To verify all the requirements are implemented correctly

We get the build at every 15 days – when the release date is near, then we get the build for every 2 or 3 days.

Identifying the scenario & writing test cases means – we write only system & integration test scenarios

Defect tracking – the defect which is reported, we track whether it is fixed by the developer or not.

In Test plan / testing methodologies – instead of **functional testing**, we must say **component testing**.

Test Bed is the other name for **test data**

The **various requirements specifications** given by the customer are,

- ≈ CRS / SRS / FS / Business Specification
- ≈ Design document
- ≈ Use cases
- ≈ Application itself

The **defect report** varies from company to company. But the following are the mandatory attributes of a defect report in all companies,

- ✓ Defect ID
- ✓ Severity
- ✓ Priority

Defect Density = (Total no. of defects found – total defects fixed) / (Total no. of TC(s) executed

Verification – Am I building the product right?

Validation – Am I building the right product?

Bugs found in Prototype testing

- **Missing Requirements** – company logo is missing in reports
- Instead of using list property, we used dropdown button or checkbox
- Links are not placed at right position
- There may be some orphan files i.e – when we click on particular link, no page is displayed
- When we click on the link, instead of sales page, report page is opening

USABILITY Testing

Testing the user friendliness of an application is called Usability testing

Let us start with an example such that we have two applications A & B which are different but doing the same job. In this, we see which one is user friendly

Given below are some of the parameters we look into for testing. In this most of them are not measurable,

- ✓ Speed
- ✓ Help
- ✓ Navigation should be simple
- ✓ Compatibility
- ✓ Look & feel
- ✓ Features
- ✓ Location of components

One important parameter other than the above said parameters is “**Effort needed to learn the application**”.

Some of them define usability testing like above written in green

Suppose in the example, we understand the software A in 2 hrs, but we take 4hrs to understand B. Let us see different cases here,

- since we understand A in 2hrs, it becomes user friendly compared to B
- suppose **look & feel** is not good for A. In this case though, we understand A in 2hrs, we cannot say that A is user friendly.
- Therefore, we look into many parameters before we say user friendliness of a software

What is LOOK & FEEL

The application should be such that it should be pleasant looking [suppose if we have red color text in yellow background, definitely we don’t feel like using it] and make a feel to the end-user to use it.

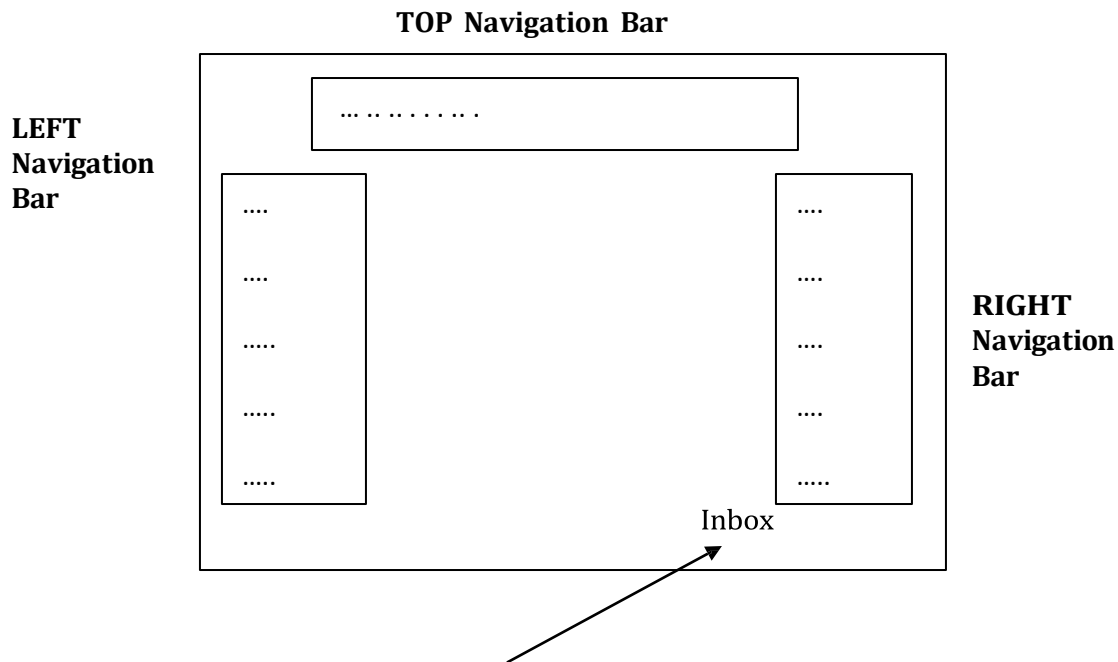
Let us now see some of the examples of Usability testing

1) Consider some of the examples below,

In this example, we see the Inbox – the most frequently used feature is not kept in the right place.

End – users therefore search for the Inbox & after sometime, they find the Inbox lying somewhere down.

This type of application may lose the customers because they are less user – friendly.



Search for INBOX – after sometime we find it here

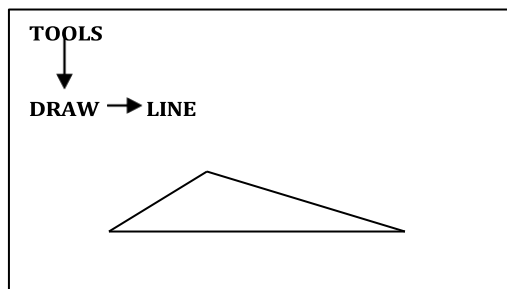
How to overcome this,

See to it that the important features are placed in the **left navigation** bar and **top navigation** bar (This is normally followed)

Then we should see whether frequently used features are easily accessible (like Inbox, Compose Mail etc)

While testing, we therefore have to list all the important features before we start testing an application

2) Consider the sketch shown below,

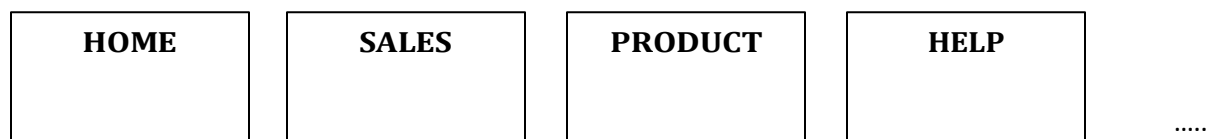


The end – user to draw the above sketch every time, he clicks **Tools -> Draw -> Line** for each line and finally he clicks **Tools -> Draw -> Line** to complete the sketch.

In this , we see the end-user wants to draw a simple sketch, but he wastes his time in clicking each time the **Tools -> Draw -> Line** for the sketch. This becomes a waste of time for the end-user.

Therefore, while testing , we must make user friendly of the application by creating **Tool bar** which makes the job simple by clicking on the necessary tools instead of going & clicking each time **Tool -> Draw -> Line**

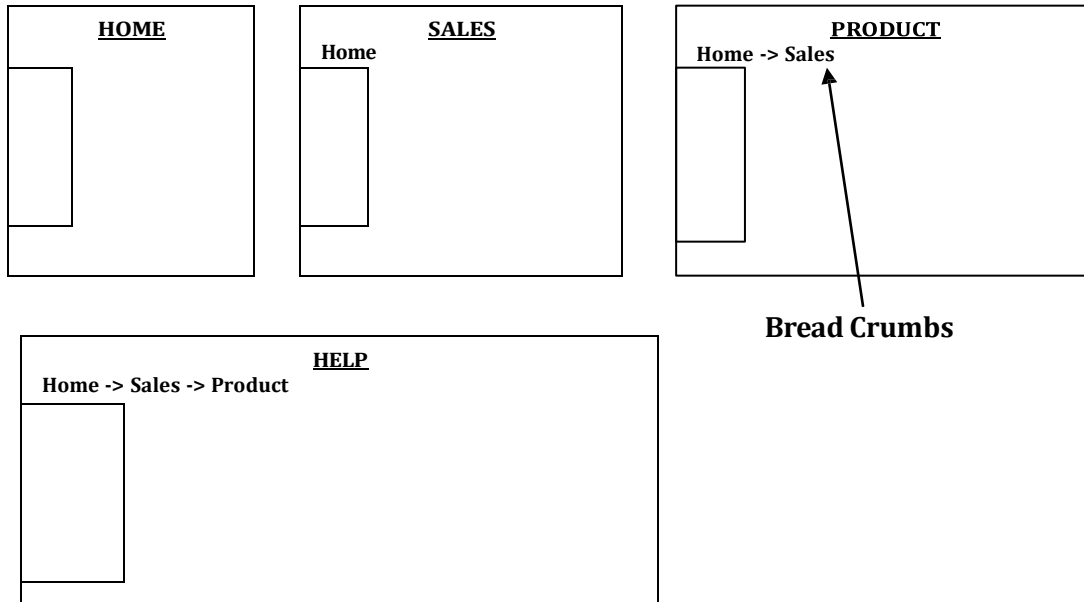
3) Suppose we have a shopping website which has 10 pages of product like shown below,



The question here is – after going to the 10th page – how do we come back to the 5th page ? – Do we click **back – after – back – back** -> NO -> then how do we do – Observe,
To make user friendly application, we always have **Bread Crumbs**

What are **Bread Crumbs** ?

In each page, we will have a link to go to its respective page. Suppose we have seen all the 7 pages & we are in 8th page – that 8th page will have the link of all the 7 pages – so whatever page (Suppose 5th page) you like to visit, you click on the link in the 8th page & go to the 5th page.



If a developer develops a **pop – up** window, always he should keep the **Yes** button in the beginning. If he swaps this, then there should be some valid reason behind it.

Generally, professional Test Engineer(s) should not do usability testing – the reason is they know where exactly the feature lies & how it works – therefore, for a TE, everything becomes user friendly in that application.

Therefore, only end user should do usability testing.

For ex,

The employee / CEO goes & collects the s/w (let us take a **game software**) & distributes to various end users like kids, friends etc. Now these end-user(s) use the games software & gives the feedback to the CEO / employee.

Now this employee / CEO looks into the feedback & sees the major feedback & consolidates all the feedback report collected from the people.

Ex – if a feature for all 50 people has been reported, then that it is considered. If a feature has been reported only by 1 or 2 people, then it becomes minor.

After consolidating, he goes for fixing the bugs.

One thing we have to keep in mind is that all the applications cannot be given to end-users & thus it depends on the application.

For ex,

In case of a **banking application** – if we develop an application for manager (call manager as end-user). For others call this respective person as end-users.

In the example above – end-users (MANAGER) starts using the application – 2 TE(s) sits at the back of him & takes report of the defect whether the end-user goes in a right way as the ydevelop as in a wrong way into the application.

Now the manager checks each link before he clicks on it because he knows the TE is watching him.

So the drawback here is – TE came to know they are not getting correct feedback.

Then how to overcome this? – do we fix camera & take feedback? -> in this , the end-user becomes more serious & thus it becomes a drawback.

80% of time, we end up doing usability testing for the reasons above mentioned.

Sometimes TE has to do usability testing for the following reasons,

- Do not want to outsource to other company
- No money to spend on Usability testing

But there are scenarios we do Usability testing where we swap the features among TE and do usability testing.

How to conduct Usability testing? (OR) What process should we follow to do Usability testing?

Prepare OR derive checklist (i.e, what are the points to be checked). If we don't prepare a checklist, we may miss some features in the application.

For Usability testing, we should prepare a genuine checklist specific to the product we develop.

Example of a checklist for an application,

- ✓ For this application, one of the checklists includes color of already checked link should be changed to red
- ✓ All the images should have **alt** tag(Tooltip)
- ✓ All the pages should have link to Homepage
- ✓ Login feature should have **Forgot Password** link

Like the above checklist, we can derive as many checklists as possible based on the application (or) product.

While deriving checklist, we should derive a common checklist which can be executed for all pages.

There is another caes where the customer gives the checklist for the application.

ACCESSIBILITY Testing / ADA (American Disability Act) / 508 Compliance testing

Testing the application from the physically challenged person point of view.

For ex, Suppose if a blind person is accessing the Internet – the application should be in such a way that even the physically challenged person should be able to access it without any problems.

When a blind person clicks on anything – the response connected into voice & the person heras & uses it – Response sent should be easily read by the browser & commented in voice.

In the above example, whatever the respond is sent to the browser should be easily read – the application should be designed like that – the respond sent should be immediately connected into voice. Thus, the blind person can use it without facing any problem

Red & Green color should not be used

The 508 testing has got some rules that should be followed while developing the application. Some of the rules are,

- ✓ All comments should have **Alt** tags
- ✓ Red & Green color objects should not be displayed
- ✓ Should be able to access all components just by using keyboard.

Like this, there are many rules.